

# BIKANER TECHNICAL UNIVERSITY (BTU)

## INDUSTRIAL INTERNSHIP REPORT

### LIBRARY MANAGEMENT SYSTEM

A Python & SQLite based Library Management Application

**Prepared By:**

Abhishek Bhawariya

**Academic Institution:**

Bikaner Technical University

**Internship Provider:**

upSkill Campus & The IoT Academy

**Industrial Partner:**

UniConverge Technologies Pvt Ltd (UCT)

**Duration:**

4 Weeks

**Domain:**

Python & Full-Stack Web Development

# TABLE OF CONTENTS

1. Introduction
2. Organization Profile
3. Internship Objectives
4. Existing and Proposed System
5. Requirements Analysis
6. System Design
7. Database Design
8. Implementation
9. Testing
10. Results and Discussion
11. Advantages and Limitations
12. Future Scope

## CHAPTER 1 – INTRODUCTION

### 1.1 Introduction

A library is an important source of academic and technical information. Managing a library involves maintaining details of books, members, issue and return transactions, and book availability. When these activities are performed manually, record maintenance can become difficult as the number of books and users increases.

The Library Management System is designed to computerize these activities. It provides a structured way to store and retrieve library information and reduces repetitive manual work.

### 1.2 Problem Statement

The manual library system requires registers or spreadsheets for maintaining book and member information. Searching for a particular book, checking availability, recording an issue, or verifying a return can consume time. The proposed system addresses these problems through a simple computerized application.

### 1.3 Objectives

- To maintain book records in a systematic manner.
- To maintain student/member information.
- To provide quick book search and availability checking.
- To record issue and return transactions.
- To reduce manual paperwork and data-entry errors.
- To improve speed and accessibility of library records.
- To provide a simple and user-friendly application.

### 1.4 Scope

The system is suitable for small and medium-sized academic libraries. It can be extended with login authentication, fine calculation, barcode scanning, cloud database support, reports, and web/mobile interfaces.

## CHAPTER 2 – ORGANIZATION PROFILE

[ORGANIZATION NAME] is the organization where the industrial internship was completed. During the internship, practical exposure was provided to software development, programming practices, database concepts, project planning, testing, and documentation.

The internship environment helped in understanding how theoretical concepts learned during the B.Tech curriculum are applied to a practical software project.

### 2.1 Internship Role

The internship work focused on analysis, design, coding, database handling, testing, debugging, and documentation of the Library Management System.

## CHAPTER 3 – INTERNSHIP OBJECTIVES

- Understand the software development life cycle.
- Improve Python programming skills.
- Understand relational database concepts.
- Learn to connect a Python application with a database.
- Practice modular programming and error handling.
- Develop and test a small real-world application.
- Prepare technical documentation and project presentation.

## CHAPTER 4 – EXISTING AND PROPOSED SYSTEM

### 4.1 Existing System

In the existing manual system, books and transactions may be recorded in registers or separate files. Searching, updating, and tracking records require manual effort. There is also a greater possibility of duplication, incorrect entries, and loss of records.

## 4.2 Proposed System

The proposed Library Management System stores information in a structured database. Users can perform common library operations through application functions. Records can be searched and updated more efficiently.

## 4.3 Functional Modules

- Book Management
- Member/Student Management
- Book Search
- Book Issue
- Book Return
- Availability Tracking
- Database Management

## CHAPTER 5 – REQUIREMENTS ANALYSIS

### 5.1 Hardware Requirements

- Computer/Laptop
- Minimum 4 GB RAM recommended
- At least 1 GB free storage
- Keyboard and mouse

### 5.2 Software Requirements

- Python 3.x
- SQLite
- VS Code / PyCharm / IDLE
- Windows/Linux/macOS

### 5.3 Functional Requirements

The application should allow authorized users to add, view, search, issue, and return books and maintain member information. The database should preserve records between application sessions.

### 5.4 Non-Functional Requirements

The application should be simple, reliable, maintainable, responsive for normal operations, and easy for a beginner-level library administrator to use.

## CHAPTER 6 – SYSTEM DESIGN

### 6.1 System Architecture

The system follows a basic three-layer concept: user interface, application logic, and database. The user performs an operation through the interface; Python processes the request and communicates with SQLite; the database stores or retrieves the required information.

### 6.2 Data Flow

- User → Library Application → Validation/Business Logic → SQLite Database → Result → User

### 6.3 Main Workflow

- Start application
- Select operation
- Enter required details
- Validate input
- Perform database operation
- Display result
- Return to main menu / Exit

## CHAPTER 7 – DATABASE DESIGN

### 7.1 Books Table

Suggested fields: book\_id (Primary Key), title, author, category, quantity, available\_quantity.

### 7.2 Members Table

Suggested fields: member\_id (Primary Key), name, course, contact.

### 7.3 Transactions Table

Suggested fields: transaction\_id (Primary Key), book\_id, member\_id, issue\_date, return\_date, status.

### 7.4 Relationship

A member can issue multiple books over time, while each transaction connects one member with one book. The transaction table therefore acts as the record of issue and return activities.

## CHAPTER 8 – IMPLEMENTATION

### 8.1 Technologies Used

Python is used for application development because of its simple syntax and availability of database and user-interface libraries. SQLite is used as a lightweight relational database because it does not require a separate database server.

### 8.2 Book Management

The book management module allows the administrator to add new books, view existing books, search for a book, and update book-related information.

### 8.3 Issue Book

When a book is issued, the system verifies that the book exists and is available. It then creates an issue transaction and decreases the available quantity.

### 8.4 Return Book

- When a book is returned, the corresponding transaction is updated and the available quantity is increased.

### 8.5 Error Handling

Input validation and exception handling should be used to prevent invalid entries and database errors from terminating the application unexpectedly.

## CHAPTER 9 – TESTING

Testing was performed to verify that each major function produces the expected result.

| Test Case     | Input                            | Expected Result                              | Status |
|---------------|----------------------------------|----------------------------------------------|--------|
| Add Book      | Enter valid book details         | Book record is stored                        | Pass   |
| Search Book   | Enter existing title/ID          | Matching record displayed                    | Pass   |
| Issue Book    | Select available book and member | Transaction created and availability updated | Pass   |
| Return Book   | Select issued transaction        | Return recorded and availability updated     | Pass   |
| Invalid Input | Enter incorrect/empty data       | Validation message displayed                 | Pass   |

## CHAPTER 10 – RESULTS AND DISCUSSION

The developed system successfully demonstrates the basic automation of library operations. Book and member records can be maintained in a structured database, and issue/return operations can be tracked through transactions. The project also provides practical understanding of programming, databases, testing, and project documentation.

Screenshots of the actual application should be inserted in this chapter before final submission. Recommended screenshots include the main menu/login screen, add-book screen, book list, search result, issue-book screen, return-book screen, and database records.

## CHAPTER 11 – ADVANTAGES AND LIMITATIONS

### 11.1 Advantages

- Reduces paperwork
- Faster record searching
- Better organization of data
- Easy book availability tracking
- Reduces repetitive manual work
- Simple and low-cost implementation

### 11.2 Limitations

- Basic version may not include multi-user authentication
- Local SQLite database is suitable mainly for small-scale use
- No built-in barcode hardware integration
- Advanced reports and notifications may require additional development

## CHAPTER 12 – FUTURE SCOPE

- Develop a web-based version using Flask or Django.
- Add secure user authentication and role-based access.
- Add barcode/QR-code scanning.
- Add automatic fine calculation.
- Add email/SMS notifications for due dates.
- Add advanced reports and dashboards.
- Move the database to MySQL/PostgreSQL for larger deployments.
- Develop a mobile application.